SOMAIYA
VIDYAVIHAR
K J Somaiya Institute of Technology

An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Name and Code: Data Structures Lab **(**ITL302)
Semester: III (SYIT)
Academic Year: 2026-27 (Odd Semester)
Name: Rashi Dawda
Roll no: 23

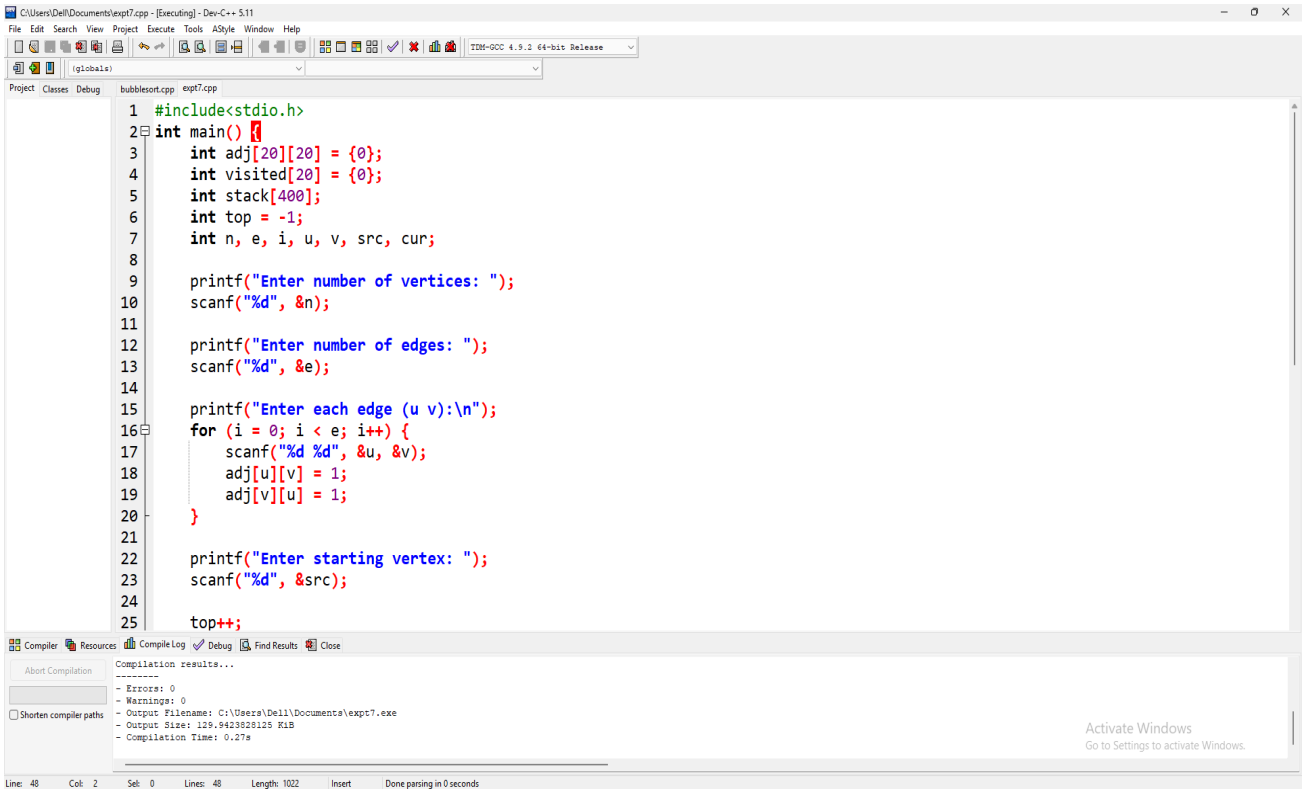
Experiment No. 07

Aim: Implementation of the Depth First Search (DFS) graph traversal technique using an adjacency matrix.

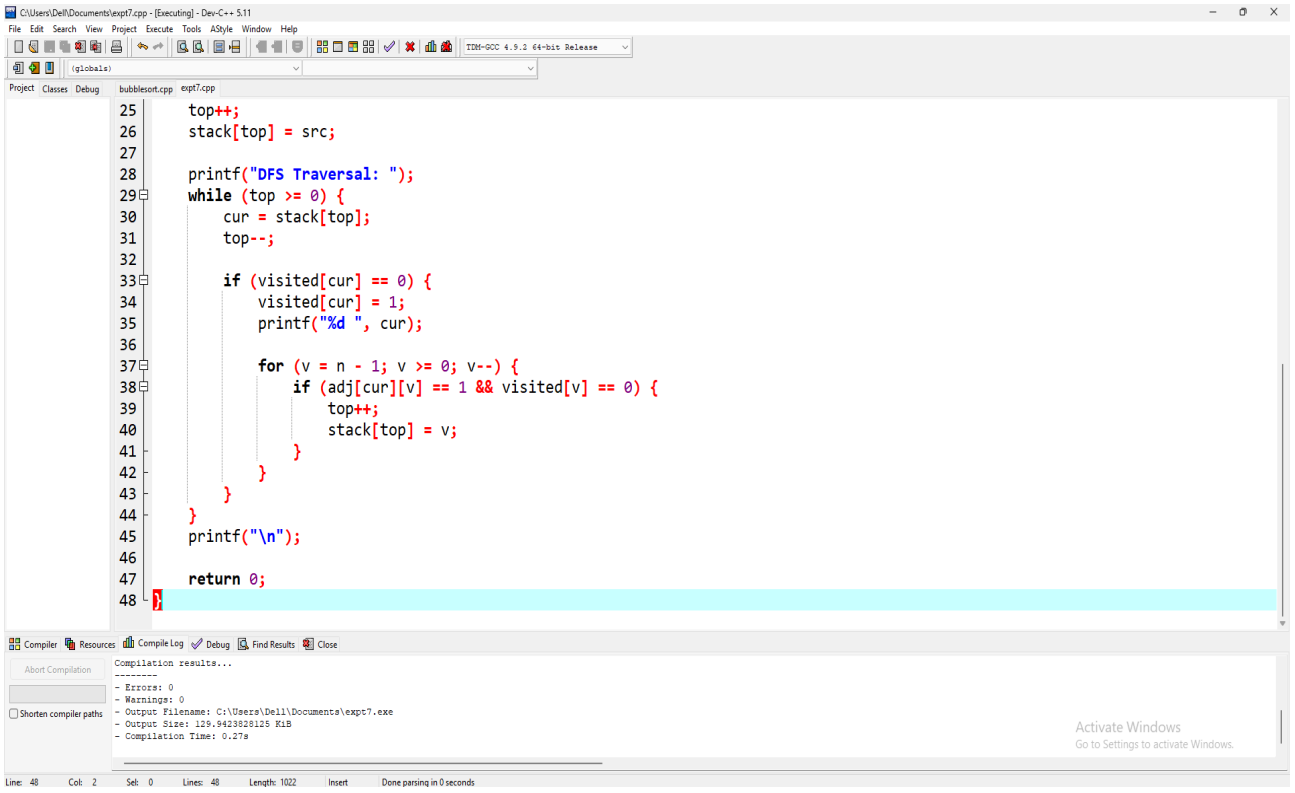
Objectives:
To introduce the concepts of data structures and analysis procedure.
To conceptualize non-linear data structures (graphs) and their traversal techniques.

Theory: Introduction to Graph
A graph G = (V, E) is a non-linear data structure made of a set of vertices V and a set of edges E that connect pairs of vertices. Graphs are of three types: directed (edges have arrows), undirected (edges have no arrows) and weighted (edges carry a cost). A graph can be stored as an adjacency matrix or as an array of adjacency lists. This experiment uses the adjacency matrix.
Depth First Search (DFS)
DFS starts at a source vertex and goes as deep as possible along one path before backtracking to explore the remaining paths. It uses a stack (LIFO) and a visited array so that no vertex is processed twice. BFS, in contrast, uses a queue.
int adj[20][20]; is the adjacency matrix. adj[u][v] = 1 if an edge exists between u and v.
int visited[20]; is the visited array. visited[i] = 1 once vertex i has been visited.
int stack[400]; int top = -1; is the stack. It holds vertices waiting to be explored.
Algorithm
Declare adj, visited, stack and set top = -1.
Read n and e. For every edge (u, v) set adj[u][v] = 1 and adj[v][u] = 1.
Read the source src and push it: top++; stack[top] = src;
While top >= 0, pop a vertex: cur = stack[top]; top--;
If visited[cur] == 0, set visited[cur] = 1 and print cur.
For v = n-1 down to 0: if adj[cur][v] == 1 and visited[v] == 0, push v.
Repeat from step 4 until the stack is empty.
Complexity
Time: $O(V^2)$, since every popped vertex scans one full row of the matrix.
Space: $O(V^2)$ for the adjacency matrix, plus O(V) for the visited array.

Code:

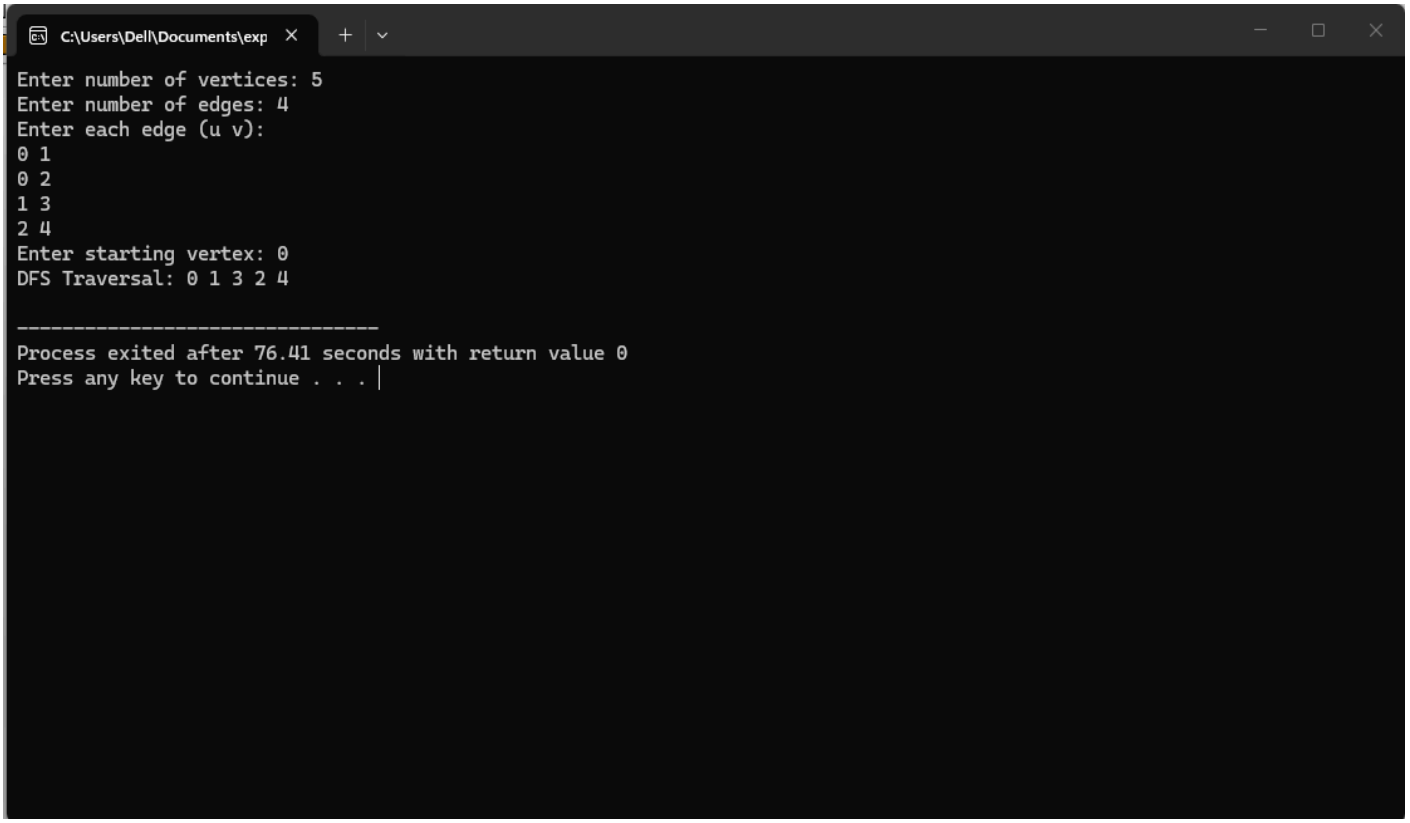

C:\Users\Dell\Documents\expt7.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
bubblesort.cpp expt7.cpp
#include<stdio.h>
int main() {
int adj[20][20] = {0};
int visited[20] = {0};
int stack[400];
int top = -1;
int n, e, i, u, v, src, cur;
printf("Enter number of vertices: ");
scanf("%d", &n);
printf("Enter number of edges: ");
scanf("%d", &e);
printf("Enter each edge (u v):\n");
for (i = 0; i < e; i++) {
scanf("%d %d", &u, &v);
adj[u][v] = 1;
adj[v][u] = 1;
}
printf("Enter starting vertex: ");
scanf("%d", &src);
top++;
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Dell\Documents\expt7.exe
- Output Size: 129.9423828125 KiB
- Compilation Time: 0.27s
Activate Windows
Go to Settings to activate Windows.
Line: 48 Col: 2 Sel: 0 Lines: 48 Length: 1022 Insert Done parsing in 0 seconds


C:\Users\Dell\Documents\expt7.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
bubblesort.cpp expt7.cpp
top++;
stack[top] = src;
printf("DFS Traversal: ");
while (top >= 0) {
cur = stack[top];
top--;
if (visited[cur] == 0) {
visited[cur] = 1;
printf("%d ", cur);
for (v = n - 1; v >= 0; v--) {
if (adj[cur][v] == 1 && visited[v] == 0) {
top++;
stack[top] = v;
}
}
}
}
printf("\n");
return 0;
}
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Dell\Documents\expt7.exe
- Output Size: 129.9423828125 KiB
- Compilation Time: 0.27s
Activate Windows
Go to Settings to activate Windows.
Line: 48 Col: 2 Sel: 0 Lines: 48 Length: 1022 Insert Done parsing in 0 seconds

Output:



Conclusion: In this experiment I learnt how DFS actually moves through a graph: it keeps going deeper along one path and only comes back when no new vertices are left. Implementing it with a stack and a visited array instead of recursion made it clear why the visited array is needed, because without it the program would keep looping in a graph that has a cycle. Tracing the sample input step by step (0 1 3 2 4) helped me match the program's output with the DFS I had worked out in my notes. The adjacency matrix keeps the code simple, but it makes the time complexity $O(V^2)$, so for bigger graphs an adjacency list would be the better choice.

Outcome: Apply the concepts of graphs and graph traversal techniques.